AI Library Seat Occupancy Tracker — Complete Project Guide

> **Prepared for:** Mini-Project Presentation & Viva

> **Last Updated:** October 28, 2026

Table of Contents

1. Project Overview
2. Problem Statement & Motivation
3. Technologies Used & Why
4. Project Folder Structure
5. System Architecture
6. Complete Data Flow
7. Backend Deep Dive — main.py
8. Frontend Deep Dive — HTML, CSS, JS
9. AI Models — YOLOv8 Explained
10. Key Algorithms
11. How to Run the Project
12. Feature Summary
13. Limitations & Future Scope
14. Viva Questions & Answers (70+)

---

1. Project Overview

Project Name: AI Library Seat Occupancy Tracker

One-Line Summary: A real-time AI system that uses a camera and YOLOv8 deep learning models to detect how many library seats are occupied and displays the results on a live web dashboard.

What It Does (Simple Explanation):

- A camera (webcam or video file) watches a library room
- AI models (YOLOv8) detect people in the video frame
- The system checks which defined "seat zones" have a person sitting in them
- A web dashboard shows: live video feed, which seats are occupied/vacant, total counts, and a visual ring chart — all updating in real-time

Key Highlight: Everything runs from a **single server** — one command starts the AI, the video processing, AND the website. No separate installations needed for the frontend.

---

2. Problem Statement & Motivation

Problem: In college and public libraries, students waste time walking around looking for empty seats, especially during exams. There's no way to know seat availability without physically going there.

Solution: Install a camera and run this AI system. Students can check a webpage from their phone/laptop to see which seats are free — in real-time.

Why AI? Traditional methods (sensors on each seat, RFID cards) are expensive and require hardware on every seat. With AI, one camera can monitor 10-20 seats simultaneously using just software.

---

3. Technologies Used & Why

3.1 Python (Programming Language)

What: A general-purpose programming language known for simplicity and readability.

Why Python for this project:

- Best ecosystem for AI/ML — libraries like PyTorch, OpenCV, Ultralytics are all Python-native
- Easy to read and write — looks almost like English
- Huge community support

3.2 FastAPI (Web Framework)

What: FastAPI is a modern Python web framework for building APIs (Application Programming Interfaces). An API is a way for different software programs to talk to each other.

Why FastAPI over alternatives:

Framework	Speed	Async Support	Auto-Docs	Type Checking	Learning Curve
FastAPI	⚡ Very Fast	✅ Built-in	✅ Automatic	✅ Pydantic	Medium
Flask	🐢 Slower	❌ Needs extensions	❌ Manual	❌ No	Easy
Django	🐢 Slower	⚠️ Partial	❌ Manual	❌ No	Hard
Express (Node.js)	⚡ Fast	✅ Built-in	❌ Manual	❌ No	Medium

Key reasons FastAPI was chosen:

1. **Async support** — Can handle video streaming, WebSockets, and AI inference simultaneously without blocking
2. **Speed** — One of the fastest Python frameworks (based on Starlette + Uvicorn)
3. **WebSocket support** — Built-in, needed for real-time data push
4. **Static file serving** — Can serve our HTML/CSS/JS frontend directly
5. **Automatic validation** — Uses Pydantic models to validate incoming data

What is "async"? Normally, a program does one thing at a time (synchronous). Async means it can start multiple tasks and switch between them while waiting. Example: while waiting for a video frame to load, it can send data to a browser. This is critical because our app does many things at once.

3.3 Uvicorn (Server)

What: Uvicorn is an ASGI server — it's the actual program that listens for web requests and runs the FastAPI application.

Analogy: If FastAPI is the restaurant kitchen (where food is prepared), Uvicorn is the restaurant building (the physical place that receives customers). You need both.

Why Uvicorn: It's the recommended server for FastAPI. It's very fast because it uses `uvloop` (a faster version of Python's default event loop).

3.4 YOLOv8 (AI Model)

What: YOLO = "You Only Look Once". It's a family of deep learning models for detecting objects in images/video. YOLOv8 is the 8th version, made by Ultralytics.

Why YOLOv8:

- **Real-time speed** — Can process 30+ frames per second
- **Pre-trained** — Already trained on 80 object classes including "person" using the COCO dataset (330,000 images)
- **Multiple variants** — We use two: Pose (skeleton keypoints) and Segmentation (body outline)
- **Easy to use** — Just 2 lines: `model = YOLO("yolov8n-pose.pt")` then `model.predict(frame)`

What is "n" in yolov8n? It means "nano" — the smallest/fastest variant. Other sizes: s (small), m (medium), l (large), x (extra-large). We use nano because speed matters more than accuracy for real-time tracking.

3.5 OpenCV (cv2)

What: Open Source Computer Vision Library. It handles everything related to images and video — reading camera frames, drawing rectangles, encoding images to JPEG, etc.

Why OpenCV: Industry standard for computer vision. No alternative comes close for video I/O and image manipulation in Python.

3.6 NumPy

What: Numerical Python — a library for fast mathematical operations on arrays/matrices.

Why: Images are stored as matrices of numbers (pixels). NumPy makes operations on these matrices very fast. OpenCV and YOLO both use NumPy internally.

3.7 HTML, CSS, JavaScript (Frontend)

What:

- **HTML** = Structure (what elements exist on the page — buttons, text, images)
- **CSS** = Style (how elements look — colors, sizes, layouts)
- **JavaScript** = Behavior (what happens when you click, how data updates in real-time)

Why plain HTML/CSS/JS instead of React:

- No build tools or npm needed — zero installation for frontend
- Every line is standard web technology taught in college
- One server serves everything (no separate frontend server)
- Examiner can read and understand the code easily
- Does the exact same job for a single-page application

3.8 WebSocket

What: A communication protocol that keeps a persistent connection between the browser and server, allowing the server to push data to the browser instantly.

Normal HTTP vs WebSocket:

- **HTTP:** Browser asks → Server responds → Connection closes. Browser must keep asking repeatedly ("polling").
- **WebSocket:** Connection stays open. Server can send data anytime without being asked. Much faster and more efficient.

Why we need it: Occupancy data changes every second. With WebSocket, the server pushes new data to the browser instantly. Without it, the browser would have to refresh or poll every second.

3.9 MJPEG Streaming

What: Motion JPEG — a video streaming method where each frame is sent as a separate JPEG image, one after another.

Why MJPEG:

- Works in any browser with a simple `<img>` tag — no video player needed
- No complex video codecs required
- Low latency — each frame is independent
- Simple to implement server-side

---

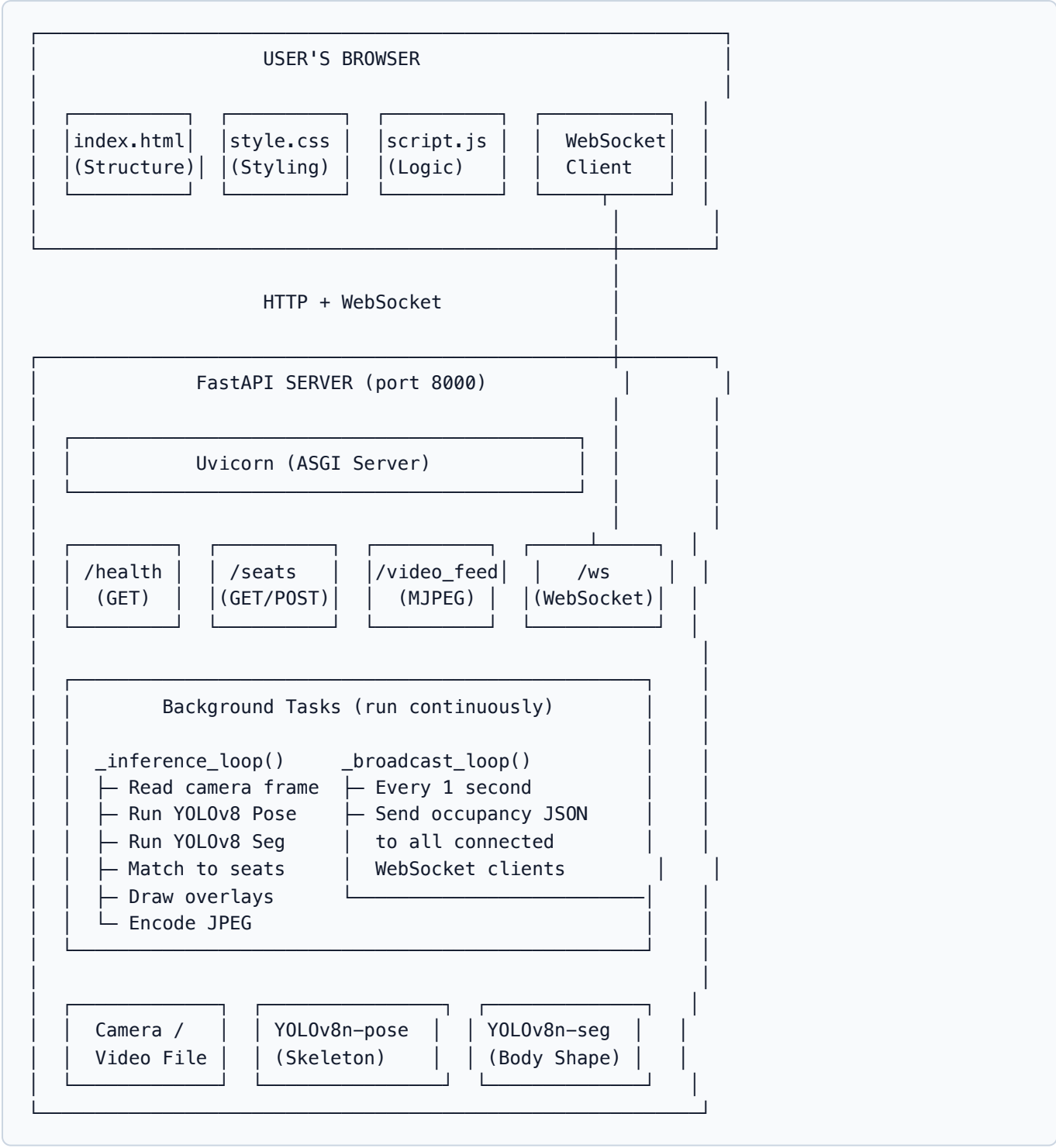
4. Project Folder Structure

```
Mini-project/
│
├── backend/                    ← All server-side code
│   ├── main.py                 ← THE main file (727 lines) — entire backend
│   ├── requirements.txt        ← Python packages needed
│   ├── seats.json              ← Saved seat zone positions
│   ├── yolov8n-pose.pt         ← AI model file (pose detection)
│   ├── yolov8n-seg.pt          ← AI model file (segmentation)
│   ├── 329307.mp4              ← Test video file (optional)
│   ├── generate_test_video.py  ← Script to create test videos
│   └── venv/                   ← Python virtual environment
│
├── frontend/                   ← All client-side code
│   ├── index.html              ← Page structure (125 lines)
│   ├── style.css               ← Page styling (300 lines)
│   └── script.js               ← Page logic (520 lines)
```

Total code: ~1,670 lines across 4 files.

---

5. System Architecture



6. Complete Data Flow

Here's exactly what happens from camera to browser, step by step:

Step 1: Camera Capture

```
cap = cv2.VideoCapture(VIDEO_SOURCE)  # Open camera (0) or video file
ret, frame = cap.read()               # Read one frame (a single image)
```

The frame is a NumPy array of shape `(height, width, 3)` — 3 channels for Blue, Green, Red.

Step 2: AI Inference (Both Models Run in Parallel)

```
seg_results, pose_results = await asyncio.gather(
    asyncio.to_thread(_run_seg),    # Segmentation model
    asyncio.to_thread(_run_pose),   # Pose model
)
```

- **Pose model** → Detects 17 body keypoints (nose, eyes, ears, shoulders, elbows, wrists, hips, knees, ankles) with confidence scores
- **Segmentation model** → Creates a pixel-perfect body outline (silhouette mask)

Step 3: Extract Torso Bounding Box

From pose keypoints, we build a tight box around ONLY the head and shoulders (keypoints 0-6). We deliberately exclude hands, elbows, knees, feet because:

- A person reaching across to a neighbor's seat shouldn't mark that seat as occupied
- The head+shoulder position is the most reliable indicator of where someone is sitting

Step 4: Match People to Seats

For each person's torso box, check every defined seat zone:

- Is the person's center inside the seat? → Strong match (overlap = 1.0)
- Otherwise, calculate how much the torso box overlaps the seat area
- Assign each person to their BEST matching seat (highest overlap)
- A person can only occupy ONE seat (prevents double-counting)

Step 5: Temporal Smoothing

If a seat was detected as occupied, hold that status for 2 seconds even if detection briefly drops. This prevents flickering when someone leans forward or turns their head momentarily.

Step 6: Draw Visual Overlays

On the video frame, draw:

- Green/red rectangles for each seat (vacant/occupied)
- Green body silhouettes for each detected person
- Labels showing "Seat 1: Occupied" etc.
- HUD showing counts ("Seats: 2/5 occupied", "People: 3")

Step 7: Encode & Store

```
_, jpeg = cv2.imencode(".jpg", annotated, [cv2.IMWRITE_JPEG_QUALITY, 75])
_latest_jpeg = jpeg.tobytes()  # Store for MJPEG streaming
```

Step 8: Update Shared State

```
latest_occupancy = {
    "zone": "Main Lab",
    "occupied_seats": 2,
    "total_seats": 5,
    "seats": [{"id": 1, "label": "Seat 1", "occupied": true}, ...],
    "person_count": 3,
    "timestamp": 1698400000.0
}
```

Step 9: Broadcast via WebSocket

Every 1 second, the broadcast loop sends `latest_occupancy` as JSON to all connected browsers.

Step 10: Browser Updates Dashboard

JavaScript receives the JSON, parses it, and updates every element on the page — ring chart, seat chips, counts, status pill, capacity bar.

---

7. Backend Deep Dive — main.py

The entire backend is a single file: `main.py` (727 lines). Here's every section explained:

7.1 Imports (Lines 10-27)

```
import asyncio          # For running multiple tasks concurrently
import json             # For reading/writing JSON data
import os               # For file path operations
import threading        # For thread-safe locks
import time             # For timestamps and delays
import cv2              # OpenCV — camera, image processing
import numpy as np      # NumPy — numerical arrays (images are arrays)
from fastapi import FastAPI, WebSocket, WebSocketDisconnect
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import StreamingResponse
from fastapi.staticfiles import StaticFiles
from pydantic import BaseModel
from ultralytics import YOLO
```

7.2 Configuration (Lines 29-64)

All tunable parameters are at the top for easy modification:

Variable	Value	Purpose
`ZONE_NAME`	"Main Lab"	Name shown on dashboard
`VIDEO_SOURCE`	`0` or `"329307.mp4"`	`0` = webcam, string = video file
`BROADCAST_INTERVAL`	1.0 sec	How often to push updates to browsers
`CONFIDENCE_THRESHOLD`	0.45	Pose model: minimum confidence to accept a detection (0-1)
`SEG_CONFIDENCE_THRESHOLD`	0.30	Seg model: lower threshold for better body outline recall
`NMS_IOU_THRESHOLD`	0.4	Non-Maximum Suppression: merges overlapping boxes
`HEAD_KP_INDICES`	[0,1,2,3,4]	COCO keypoints: nose, left_eye, right_eye, left_ear, right_ear
`HEAD_KP_CONFIDENCE`	0.3	Minimum keypoint confidence
`MIN_HEAD_KPS`	2	At least 2 head keypoints must be visible
`OVERLAP_THRESHOLD`	0.15	Minimum overlap fraction to count as "occupied"
`OCCUPANCY_HOLD_SECONDS`	2.0	Temporal smoothing hold time

7.3 Data Models (Lines 66-113)

SeatZoneIn — Pydantic model for validating incoming seat data:

```
class SeatZoneIn(BaseModel):
    id: int          # Unique seat number
    label: str       # Display name ("Seat 1")
    x1: float        # Top-left X (0 to 1, normalized)
    y1: float        # Top-left Y
    x2: float        # Bottom-right X
    y2: float        # Bottom-right Y
```

Why normalized coordinates (0-1)? The actual video resolution might be 1920×1080 or 640×480 — different cameras give different sizes. By storing coordinates as fractions (e.g., 0.25 means "25% from the left"), the seat zones work regardless of resolution.

SeatZoneState — Runtime state that adds `occupied` boolean:

```
class SeatZoneState:
    # ... all fields from SeatZoneIn plus:
    self.occupied = False  # Updated by inference loop
```

7.4 Persistence — seats.json (Lines 116-136)

Seat definitions are saved to `seats.json` so they survive server restarts:

```
def _save_seats(seats):
    with open(SEATS_FILE, "w") as f:
        json.dump([s.to_defn() for s in seats], f, indent=2)

def _load_seats():
    with open(SEATS_FILE, "r") as f:
        raw = json.load(f)
    return [SeatZoneState(SeatZoneIn(**s)) for s in raw]
```

7.5 Shared State & Thread Safety (Lines 138-158)

```
seat_zones: List[SeatZoneState] = _load_seats()
_seat_lock = threading.Lock()       # Protects seat_zones from race conditions
_frame_lock = threading.Lock()      # Protects _latest_jpeg
_latest_jpeg: bytes | None = None   # Most recent annotated frame
connected_clients: Set[WebSocket] = set()  # All connected browsers
```

Why locks? The inference loop runs in a background thread while the web server runs in the main thread. If both try to read/write `seat_zones` at the same time, data could get corrupted. A `Lock` ensures only one thread accesses the data at a time.

7.6 Person Contour Merging — Union-Find (Lines 162-242)

Problem: Sometimes the segmentation model splits one person into two blobs (e.g., if an arm is extended away from the body).

Solution: Union-Find algorithm (also called Disjoint Set Union):

1. For every pair of detected person blobs, calculate how much their bounding boxes overlap
2. If overlap (IoU) > 15% OR one box contains > 40% of the other → merge them
3. Use Union-Find data structure for efficient grouping
4. Combine merged blobs into a single convex hull

What is Union-Find? A data structure that efficiently groups elements. Each element has a "parent". To check if two elements are in the same group, follow parents to the root. To merge groups, point one root to the other. Uses "path compression" for speed.

7.7 Overlap Detection (Lines 245-322)

Two methods for checking if a person is in a seat:

Method 1: Polygon-based (using segmentation contour)

```
def _polygon_seat_overlap_ratio(contour, sx1, sy1, sx2, sy2):
    # Create a small image (mask) the size of the seat
    # Draw the person's body shape on it
    # Count how many pixels overlap
    # Return: overlap_pixels / seat_total_pixels
```

Method 2: Rectangle-based (using bounding box)

```
def _overlap_ratio(px1, py1, px2, py2, sx1, sy1, sx2, sy2):
    # Calculate intersection area of two rectangles
    # Return: intersection_area / seat_area
```

Method 3: Center-in-rect check

```
def _center_in_rect(px1, py1, px2, py2, sx1, sy1, sx2, sy2):
    # Calculate center point of person box
    # Return True if center is inside seat rectangle
```

7.8 The Inference Loop — Heart of the System (Lines 325-579)

This is the most important function. It runs continuously as a background task:

```
LOOP (runs every ~50ms = 20 frames per second):
│
├── 1. Read frame from camera
│
├── 2. Run BOTH AI models in PARALLEL:
│   ├── Pose model → 17 keypoints per person
│   └── Seg model  → body outline per person
│
├── 3. From Pose: extract head+shoulder keypoints
│   ├── At least 2 head keypoints must be visible
│   └── Build tight "torso box" from keypoints 0–6
│
├── 4. From Seg: extract body contours
│   └── Only keep contours that match a pose-validated person
│
├── 5. For each person's torso box:
│   ├── Check every seat zone
│   ├── Find the seat with highest overlap
│   └── Mark that ONE seat as occupied
│
├── 6. Apply temporal smoothing (2-sec hold)
│
├── 7. Draw on frame:
│   ├── Seat rectangles (green=vacant, red=occupied)
│   ├── Person silhouettes (green outline)
│   ├── Labels ("Seat 1: Occupied")
│   └── HUD counts
│
├── 8. Encode frame to JPEG → store in _latest_jpeg
│
└── 9. Update latest_occupancy dictionary
```

Why two models?

- **Pose model** is used for SEAT DETECTION (head+shoulders tell us WHERE someone is sitting)
- **Seg model** is used for VISUAL DISPLAY (body silhouette looks nice on the video feed)
- They serve different purposes and complement each other

7.9 Broadcast Loop (Lines 582-598)

Simple loop that runs every 1 second:

```
async def _broadcast_loop():
    while True:
        await asyncio.sleep(BROADCAST_INTERVAL)
        payload = json.dumps(latest_occupancy)
        for ws in connected_clients:
            await ws.send_text(payload)  # Push to each browser
```

7.10 FastAPI Lifespan (Lines 601-613)

Controls what happens when the server starts and stops:

```
@asynccontextmanager
async def lifespan(app):
    # STARTUP: Start background tasks
    inference_task = asyncio.create_task(_inference_loop())
    broadcast_task = asyncio.create_task(_broadcast_loop())
    yield  # Server is now running...
    # SHUTDOWN: Cancel tasks, release camera
    inference_task.cancel()
    broadcast_task.cancel()
```

7.11 API Endpoints (Lines 616-727)

Endpoint	Method	Purpose
`/health`	GET	Returns `{"status": "ok"}` — used to check if server is alive
`/seats`	GET	Returns all seat zone definitions
`/seats`	POST	Saves new seat zones (from calibrator)
`/seats/{id}`	DELETE	Remove a single seat zone
`/video_feed`	GET	MJPEG stream — continuous JPEG frames
`/occupancy`	GET	Returns current occupancy snapshot
`/ws`	WebSocket	Persistent connection for real-time updates
`/`	Static	Serves `index.html`, `style.css`, `script.js`

CORS Middleware:

```
app.add_middleware(CORSMiddleware, allow_origins=["*"], ...)
```

CORS = Cross-Origin Resource Sharing. This allows the frontend to call API endpoints even if served from a different domain. We allow all origins (`*`) for simplicity.

---

8. Frontend Deep Dive — HTML, CSS, JS

8.1 index.html (125 lines) — Page Structure

The HTML defines WHAT exists on the page. Key sections:

Header:

```
<header>
  <h1>Library Seat Occupancy Tracker</h1>
  <span id="clock">--:--:-- --</span>           <!-- Live clock -->
  <span id="connection-badge">● Connecting…</span>  <!-- WebSocket status -->
</header>
```

Video Panel (left side):

```
<section class="video-panel">
  <img id="video-feed" src="/video_feed" />     <!-- MJPEG stream -->
  <canvas id="cal-canvas"></canvas>              <!-- Seat drawing overlay -->
  <button onclick="toggleCalibration()">⚙ Setup Seats</button>
</section>
```

Stats Panel (right side):

```
<section class="stats-panel">
  <!-- SVG Ring chart -->
  <svg><circle class="ring-fill" r="75" /></svg>

  <!-- Counts -->
  <strong id="occ-count">0</strong> / <span id="total-count">0</span>

  <!-- Status pill: "Available" / "Full" -->
  <div id="status-pill">No Seats Defined</div>

  <!-- Capacity bar -->
  <div class="capacity-fill" id="cap-fill"></div>

  <!-- Individual seat chips -->
  <div id="seat-grid"></div>
</section>
```

How does `<img src="/video_feed">` show video?

The `/video_feed` endpoint returns a never-ending stream of JPEG images (MJPEG). The browser's `<img>` tag keeps receiving and displaying new images, creating the appearance of a video. No JavaScript is needed for this — the browser handles it natively.

8.2 style.css (300 lines) — Visual Design

Key CSS concepts used:

CSS Variables (defining colors once, using everywhere):

```
:root {
  --green: #16a34a;
  --red: #dc2626;
  --amber: #d97706;
  --gray: #6b7280;
}
/* Usage: */
.btn-green { color: var(--green); }
```

CSS Grid (two-column layout):

```
.main {
  display: grid;
  grid-template-columns: 1fr 340px;  /* Left takes remaining space, right is 340px */
  gap: 12px;
}
```

SVG Ring Chart Styling:

```
.ring-fill {
  stroke-dasharray: 471.24;     /* Full circumference = 2 × π × 75 */
  stroke-dashoffset: 471.24;    /* Start empty (offset = full circumference) */
  /* To show 50% filled: offset = 471.24 × 0.5 = 235.62 */
}
```

Responsive Design (stacks vertically on small screens):

```
@media (max-width: 840px) {
  .main { grid-template-columns: 1fr; }  /* Single column on mobile */
}
```

8.3 script.js (520 lines) — All Frontend Logic

Section 1: DOM References (Lines 6-30)

```
var clockEl = document.getElementById("clock");
var badgeEl = document.getElementById("connection-badge");
// ... 16 more element references
```

`document.getElementById()` finds an HTML element by its `id` attribute so JavaScript can modify it later.

Section 2: WebSocket Connection (Lines 64-108)

```
function connectWebSocket() {
  var wsUrl = "ws://" + window.location.host + "/ws";
  ws = new WebSocket(wsUrl);

  ws.onopen = function() {
    badgeEl.textContent = "● Live";        // Show "Live" badge
  };

  ws.onmessage = function(event) {
    var data = JSON.parse(event.data);      // Parse JSON string to object
    updateDashboard(data);                  // Update all UI elements
  };

  ws.onclose = function() {
    setTimeout(connectWebSocket, 3000);     // Auto-reconnect after 3 seconds
  };
}
```

Section 3: Dashboard Update (Lines 111-207)

When new data arrives via WebSocket, `updateDashboard()` updates:

- Zone name text
- Occupied/total counts
- Ring progress (SVG stroke-dashoffset animation)
- Status pill color and label
- Capacity bar width and color
- Seat grid chips (green "Vacant" or red "Occupied")

Ring Progress Math:

```
var circumference = 2 * Math.PI * 75;           // = 471.24
var ratio = occupied / total;                     // e.g., 3/5 = 0.6
var offset = circumference * (1 - ratio);         // = 471.24 × 0.4 = 188.5
ringFill.style.strokeDashoffset = offset;         // Shows 60% of ring filled
```

Section 4: Seat Calibration (Lines 238-480)

The calibrator lets users define seat zones by drawing rectangles on the video:

1. **Enter calibration mode:** Click "Setup Seats" → Canvas overlay appears on top of video
2. **Draw a seat:** Click and drag → Blue dashed rectangle follows mouse
3. **Release:** If rectangle is big enough (>2% of frame), create a new seat
4. **Delete:** Each seat has a small red × button — click to remove
5. **Undo:** Remove the last-drawn seat
6. **Clear All:** Remove all seats
7. **Auto-save:** Every change immediately calls `POST /seats` to save to the backend

Normalized coordinates: All seat positions are stored as 0-1 fractions:

```
var pos = {
  x: (event.clientX - rect.left) / rect.width,   // 0 to 1
  y: (event.clientY - rect.top) / rect.height     // 0 to 1
};
```

Section 5: Seat API Calls (Lines 483-520)

```
// Save seats to backend
fetch("/seats", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ seats: seats })
});

// Load seats on page load
fetch("/seats")
  .then(function(res) { return res.json(); })
  .then(function(data) { seats = data.seats; });
```

---

9. AI Models — YOLOv8 Explained

9.1 What is Deep Learning?

- **Traditional programming:** Human writes rules → Computer follows rules
- **Machine Learning:** Human provides data → Computer learns patterns
- **Deep Learning:** A type of ML using "neural networks" with many layers (hence "deep")

9.2 What is YOLO?

YOLO (You Only Look Once) is a real-time object detection algorithm:

- **Input:** An image (e.g., 640×480 pixels)
- **Output:** Bounding boxes with class labels and confidence scores

How it works (simplified):

1. Divide the image into a grid (e.g., 20×20)
2. For each grid cell, predict: "Is there an object here? What class? Where exactly?"
3. Filter out low-confidence predictions
4. Apply NMS (Non-Maximum Suppression) to remove duplicate detections
5. Output final list of detections

Why "You Only Look Once"? Earlier methods (R-CNN) scanned the image multiple times. YOLO processes the entire image in a single pass through the neural network — much faster.

9.3 Two Models We Use

Model 1: YOLOv8n-Pose (yolov8n-pose.pt)

- Detects people AND their skeleton (17 keypoints)
- We use keypoints 0-6 (head + shoulders) for seat detection
- Used for: DECIDING which seats are occupied

Model 2: YOLOv8n-Seg (yolov8n-seg.pt)

- Detects people AND their pixel-perfect body outline
- Produces a segmentation mask (which pixels belong to the person)
- Used for: DRAWING body silhouettes on the video

9.4 COCO Keypoints (17 total)

```
 0: Nose
 1: Left Eye       2: Right Eye
 3: Left Ear       4: Right Ear
 5: Left Shoulder  6: Right Shoulder
 7: Left Elbow     8: Right Elbow
 9: Left Wrist    10: Right Wrist
11: Left Hip      12: Right Hip
13: Left Knee     14: Right Knee
15: Left Ankle    16: Right Ankle
```

We use 0-6 (head region) for seat detection because the head position reliably indicates where someone is sitting.

9.5 Non-Maximum Suppression (NMS)

Problem: The model might detect the same person 3 times with slightly different boxes.

Solution (NMS):

1. Sort all boxes by confidence (highest first)
2. Take the best box → keep it
3. Remove all other boxes that overlap > threshold (IoU > 0.4) with the kept box
4. Repeat until no boxes remain

9.6 What is IoU?

IoU = Intersection over Union

```
IoU = Area of Overlap / Area of Combined Region
```

- IoU = 1.0 → Perfect overlap (same box)
- IoU = 0.5 → 50% overlap
- IoU = 0.0 → No overlap at all

---

10. Key Algorithms

10.1 Temporal Smoothing

```
OCCUPANCY_HOLD_SECONDS = 2.0

if detected_now:
    seat_last_occupied[seat.id] = now     # Record timestamp

last_seen = seat_last_occupied.get(seat.id, 0.0)
occupied = (now - last_seen) < OCCUPANCY_HOLD_SECONDS  # True if within 2 sec
```

Why? AI detection isn't perfect. If someone turns their head for 0.5 seconds, detection might briefly fail. Without smoothing, the seat would flicker between "occupied" and "vacant". The 2-second hold prevents this.

10.2 Exclusive Seat Assignment

```
for person in torso_boxes:
    best_seat = find_highest_overlap_seat(person)
    mark_occupied(best_seat)  # ONE person → ONE seat only
```

Why? If a person sits between two seats, their bounding box might overlap both. Without exclusive assignment, both seats would show as "occupied" — wrong! We assign each person to their single best-matching seat.

10.3 Head Keypoint Verification

```
head_visible = count(keypoints[0:4] with confidence > 0.3)
if head_visible < 2:
    skip_this_detection()  # Not a real person
```

Why? The model sometimes detects random objects as "person" (false positives). Real people have visible heads. By requiring at least 2 head keypoints (nose, eyes, ears), we filter out most false detections.

10.4 Parallel Model Execution

```
seg_results, pose_results = await asyncio.gather(
    asyncio.to_thread(_run_seg),
    asyncio.to_thread(_run_pose),
)
```

What this does: Runs both AI models at the same time on separate CPU threads. Without this, they'd run one after another, doubling the processing time.

asyncio.to_thread() moves CPU-heavy work (AI inference) to a background thread so it doesn't block the web server from handling requests.

---

11. How to Run the Project

Prerequisites

- Python 3.9+
- A webcam (or use the included test video)

Step-by-Step

```
# 1. Navigate to the project
cd Mini-project/backend

# 2. Create virtual environment (first time only)
python3 -m venv venv

# 3. Activate virtual environment
source venv/bin/activate        # macOS/Linux
# venv\Scripts\activate         # Windows

# 4. Install dependencies (first time only)
pip install -r requirements.txt

# 5. Start the server
uvicorn main:app --reload --host 0.0.0.0 --port 8000

# 6. Open browser
# Go to: http://localhost:8000
```

To Stop the Server

Press `Ctrl + C` in the terminal. This triggers the FastAPI lifespan shutdown, which:

1. Cancels the inference loop
2. Cancels the broadcast loop
3. Releases the camera (`cap.release()`)

To Switch Between Webcam and Video File

In `main.py`, line 35:

```
VIDEO_SOURCE = 0              # Use webcam
VIDEO_SOURCE = "329307.mp4"   # Use test video file
```

---

12. Feature Summary

#	Feature	Technology
1	Real-time person detection	YOLOv8-Pose
2	Body silhouette visualization	YOLOv8-Seg
3	Seat zone calibration (draw on video)	HTML Canvas + JS
4	Real-time WebSocket updates (1/sec)	FastAPI WebSocket
5	Live MJPEG video streaming	FastAPI StreamingResponse
6	SVG ring progress chart	SVG + CSS animation
7	Individual seat status chips	JavaScript DOM manipulation
8	Capacity bar with color coding	CSS transitions
9	Status pill (Available/Nearing/Full)	JavaScript logic
10	Auto-reconnect on disconnect	JavaScript setTimeout
11	Temporal smoothing (anti-flicker)	Python timestamp logic
12	Exclusive seat assignment	Python greedy algorithm
13	Person contour merging	Union-Find algorithm
14	Persistent seat storage	JSON file I/O
15	Responsive design (mobile-friendly)	CSS Grid + Media Queries
16	Live clock and date display	JavaScript setInterval
17	Head keypoint verification	COCO keypoint filtering
18	Parallel model execution	asyncio.gather + threading

---

13. Limitations & Future Scope

Current Limitations

1. **Single camera** — Only monitors one room/angle
2. **Lighting dependent** — Poor lighting reduces detection accuracy
3. **No authentication** — Anyone on the network can access the dashboard
4. **No database** — No historical data storage (only real-time)
5. **CPU-bound** — Runs on CPU; GPU would be faster

Future Scope

1. **Multi-camera support** — Monitor multiple rooms

2. **Mobile app** — Native Android/iOS app with push notifications
3. **Historical analytics** — Track occupancy trends over hours/days/weeks
4. **Booking system** — Allow students to reserve seats
5. **GPU acceleration** — Use CUDA for faster inference
6. **User authentication** — Login system for admins
7. **Cloud deployment** — Host on AWS/GCP for remote access
8. **Email/SMS alerts** — Notify when capacity reaches threshold

---

14. Viva Questions & Answers

Category 1: Project Overview

Q1: What is this project about?

A: It's an AI-powered system that uses a camera and YOLOv8 deep learning models to detect which library seats are occupied in real-time and displays the results on a web dashboard.

Q2: What problem does it solve?

A: Students waste time walking around the library looking for empty seats. This system lets them check seat availability from their phone/laptop before going.

Q3: What are the main technologies used?

A: Python, FastAPI (web server), YOLOv8 (AI object detection), OpenCV (video processing), and plain HTML/CSS/JavaScript (frontend dashboard).

Q4: How many servers does this project need?

A: Just one. FastAPI serves both the API endpoints (video, WebSocket, seat management) AND the frontend HTML/CSS/JS files. One command starts everything.

Q5: How is this different from using sensors on each seat?

A: Sensors require physical hardware on every seat (expensive, maintenance-heavy). Our solution uses a single camera and software — much cheaper and easier to deploy.

Category 2: FastAPI & Backend

Q6: What is FastAPI?

A: FastAPI is a modern, high-performance Python web framework for building APIs. It supports async operations, WebSockets, automatic data validation, and static file serving.

Q7: Why FastAPI instead of Flask or Django?

A: FastAPI has built-in async support (critical for video streaming + WebSocket + AI running simultaneously), built-in WebSocket support, automatic request validation with Pydantic, and is significantly faster than Flask/Django.

Q8: What is an API?

A: API (Application Programming Interface) is a way for software programs to communicate. Our API lets the browser request video feeds (`/video_feed`), occupancy data (`/occupancy`), and seat configurations (`/seats`).

Q9: What is Uvicorn?

A: Uvicorn is an ASGI server that runs the FastAPI application. It listens on port 8000 for incoming requests and routes them to FastAPI. Think of it as the "engine" that powers the web server.

Q10: What is ASGI?

A: ASGI (Asynchronous Server Gateway Interface) is a standard for Python async web servers. It replaced the older WSGI standard which couldn't handle WebSockets or async operations.

Q11: What does `--reload` flag do in uvicorn?

A: It watches for file changes and automatically restarts the server. Useful during development — you don't have to manually stop and restart after each code change.

Q12: What does `--host 0.0.0.0` mean?

A: It tells the server to listen on all network interfaces, so other devices on the same WiFi can access it. Without it, only `localhost` (the same computer) can connect.

Q13: What is the lifespan function?

A: It's FastAPI's way of running code on server startup and shutdown. We use it to start the inference and broadcast background tasks on startup, and cancel them on shutdown.

Q14: What is CORS and why is it needed?

A: CORS (Cross-Origin Resource Sharing) is a browser security feature that blocks requests between different domains. We enable it so the frontend (even if served from a different origin during development) can call our API.

Q15: How are seat zones persisted?

A: They're saved as a JSON file (`seats.json`). When seats are created/modified via the calibrator, a `POST /seats` request saves them to disk. On server restart, `_load_seats()` reads them back.

Category 3: AI & YOLOv8

Q16: What is YOLO?

A: YOLO (You Only Look Once) is a real-time object detection algorithm. It processes an entire image in a single forward pass through a neural network, making it very fast compared to older multi-pass approaches.

Q17: Why YOLOv8 specifically?

A: YOLOv8 (by Ultralytics) offers multiple task variants (detection, pose, segmentation) in a single framework, is state-of-the-art in accuracy, and is very easy to use with just 2 lines of Python code.

Q18: What does the "n" in yolov8n mean?

A: "Nano" — the smallest and fastest model variant. Other sizes are s(mall), m(edium), l(arge), x(tra-large). We use nano because real-time speed matters more than maximum accuracy.

Q19: Why do you use TWO models?

A: The Pose model detects body keypoints (head, shoulders) — we use these for accurate seat detection. The Segmentation model creates body outlines — we use these for visual display on the video. They serve different purposes.

Q20: What is the COCO dataset?

A: COCO (Common Objects in Context) is a large-scale dataset with 330,000+ images and 80 object classes. YOLOv8 is pre-trained on COCO, so it already knows how to detect "person" out of the box.

Q21: What are keypoints in pose detection?

A: Keypoints are specific body landmarks — the COCO format defines 17: nose, eyes, ears, shoulders, elbows, wrists, hips, knees, ankles. Each keypoint has an (x, y) position and confidence score.

Q22: Why do you only use head+shoulder keypoints for seat detection?

A: Hands, elbows, and feet can extend into neighboring seats. The head and shoulders stay above the actual seat, making them the most reliable indicator of where someone is sitting.

Q23: What is NMS (Non-Maximum Suppression)?

A: An algorithm that removes duplicate detections. If the model detects the same person 3 times with overlapping boxes, NMS keeps only the highest-confidence one and removes the rest.

Q24: What is IoU?

A: Intersection over Union — a metric for measuring overlap between two bounding boxes. IoU = overlap area ÷ combined area. Values range from 0 (no overlap) to 1 (perfect overlap).

Q25: What is a confidence threshold?

A: The minimum confidence score (0 to 1) for accepting a detection. Our pose model uses 0.45 — detections below 45% confidence are discarded as likely false positives.

Q26: What is segmentation vs detection?

A: Detection gives a bounding box (rectangle around the object). Segmentation gives a pixel-perfect mask (exactly which pixels belong to the object). Segmentation is more precise but computationally heavier.

Q27: Can this work with other AI models?

A: Yes. The architecture is model-agnostic. You could swap YOLOv8 for any detection model (like EfficientDet, DETR) as long as it outputs bounding boxes or keypoints.

Category 4: Computer Vision & OpenCV

Q28: What is OpenCV?

A: Open Source Computer Vision Library — the industry-standard library for image/video processing. We use it for camera capture, drawing overlays, encoding JPEG, and creating segmentation masks.

Q29: How does the MJPEG stream work?

A: The server sends an endless stream of JPEG images with special headers (`multipart/x-mixed-replace`). The browser's `<img>` tag understands this format and continuously displays each new frame.

Q30: What does `cv2.imencode` do?

A: Converts a NumPy image array into JPEG bytes. We use it to convert each annotated frame into a format the browser can display.

Q31: What is `cv2.addWeighted` ?

A: Blends two images together with transparency. We use it to create semi-transparent seat zone overlays (25% opacity) and body silhouette fills (15% opacity).

Q32: How are seat zones drawn on the video?

A: Using OpenCV drawing functions — `cv2.rectangle()` for seat boxes, `cv2.putText()` for labels, and `cv2.fillPoly()` for body silhouettes. These are drawn on a copy of the frame before encoding to JPEG.

Category 5: Frontend & JavaScript

Q33: How does the frontend get real-time data?

A: Through WebSocket. When the page loads, JavaScript opens a persistent WebSocket connection to `ws://localhost:8000/ws`. The server pushes occupancy data (JSON) every second. JavaScript parses it and updates all UI elements.

Q34: What is the difference between HTTP and WebSocket?

A: HTTP is request-response (client asks, server answers, connection closes). WebSocket is persistent bidirectional (connection stays open, server can push data anytime without being asked).

Q35: How does the ring progress chart work?

A: It's an SVG circle with `stroke-dasharray` (total circumference = 471.24px) and `stroke-dashoffset`. To show 60% filled: offset = 471.24 × 0.4 = 188.5. CSS transitions animate the change smoothly.

Q36: How does the seat calibrator work?

A: An HTML5 Canvas element is overlaid on the video. Mouse events (mousedown, mousemove, mouseup) track drag gestures. Coordinates are normalized to 0-1 fractions. On mouseup, a seat definition is created and sent to the backend via `POST /seats`.

Q37: What is `document.getElementById()`?

A: A JavaScript method that finds an HTML element by its `id` attribute. We use it to get references to elements we need to update (like the clock, badge, counts, etc.).

Q38: What does `JSON.parse()` do?

A: Converts a JSON string (text) into a JavaScript object that we can access with dot notation. Example: `JSON.parse('{"name":"Seat 1"}')` → `{name: "Seat 1"}`.

Q39: What is `fetch()` in JavaScript?

A: A built-in browser function for making HTTP requests. We use it to save seats (`POST /seats`) and load seats (`GET /seats`) from the backend.

Q40: How does auto-reconnect work?

A: When the WebSocket connection closes (`ws.onclose`), we use `setTimeout(connectWebSocket, 3000)` to try reconnecting after 3 seconds. This repeats until the connection succeeds.

Q41: Why plain HTML/CSS/JS instead of React?

A: For a single-page college project, plain web technologies are sufficient and much simpler — no npm, no build tools, no framework installation. The examiner can read every line directly. React would be overkill here.

Category 6: Concurrency & Threading

Q42: What is async/await?

A: A Python feature for concurrent programming. `async` defines a function that can be paused. `await` pauses execution until an operation completes, while allowing other async tasks to run during the wait.

Q43: Why is async important for this project?

A: The server must handle multiple things simultaneously — serving video, processing AI inference, broadcasting WebSocket updates, and responding to HTTP requests. Without async, each operation would block all others.

Q44: What is `asyncio.gather()`?

A: Runs multiple async tasks simultaneously and waits for all to complete. We use it to run both AI models (pose + seg) in parallel, cutting processing time nearly in half.

Q45: What is `asyncio.to_thread()`?

A: Moves a blocking function (like AI inference) to a background thread so it doesn't block the async event loop. Without this, the web server would freeze during each AI inference.

Q46: What is a threading.Lock?

A: A synchronization mechanism that prevents two threads from accessing shared data simultaneously. We have two locks: `_seat_lock` (protects seat zone data) and `_frame_lock` (protects the latest JPEG frame).

Q47: What is a race condition?

A: A bug that occurs when two threads access shared data at the same time. Example: Thread A reads seat data while Thread B is halfway through modifying it → corrupted data. Locks prevent this.

Category 7: Algorithms

Q48: What is temporal smoothing?

A: A technique to prevent rapid flickering. When a seat is detected as occupied, we "hold" that status for 2 seconds even if detection briefly fails. This smooths out momentary detection gaps.

Q49: Explain the Union-Find algorithm.

A: Union-Find (Disjoint Set Union) efficiently groups elements into sets. Each element has a parent pointer. `Find(x)` follows parent pointers to the root. `Union(a,b)` merges two groups. We use it to merge split person detections. Time complexity: nearly O(1) per operation with path compression.

Q50: What is path compression in Union-Find?

A: An optimization where, during `Find(x)`, we update each visited node to point directly to the root. This flattens the tree, making future lookups faster.

Q51: Why exclusive seat assignment?

A: If a person sits between two seats, their bounding box overlaps both. Without exclusive assignment, both seats would incorrectly show as occupied. We assign each person to only their highest-overlap seat.

Q52: How does normalized coordinate system work?

A: Instead of pixel coordinates (e.g., x=500), we store fractions (e.g., x=0.52 meaning 52% from the left). This makes seat zones resolution-independent — they work whether the video is 640×480 or 1920×1080.

Category 8: Networking & Protocols

Q53: What is HTTP?

A: HyperText Transfer Protocol — the standard protocol for web communication. Client sends a request (GET, POST, etc.), server sends a response. Each exchange is independent (stateless).

Q54: What is a REST API?

A: REST (Representational State Transfer) is an architecture style for APIs. It uses standard HTTP methods: GET (read), POST (create), PUT (update), DELETE (remove). Our `/seats` endpoint follows REST conventions.

Q55: What is JSON?

A: JavaScript Object Notation — a lightweight data format for exchanging data. Uses key-value pairs: `{"name": "Seat 1", "occupied": true}`. Both Python and JavaScript can easily read/write JSON.

Q56: What port is used and why?

A: Port 8000. A port is like a door number on a building (the server). Port 80 is default for HTTP, 443 for HTTPS, 8000 is commonly used for development servers. Multiple apps can run on the same machine using different ports.

Q57: What does `localhost` mean?

A: It refers to the current computer itself (IP 127.0.0.1). When we access `http://localhost:8000`, the browser connects to the server running on the same machine.

Category 9: Software Design Patterns

Q58: What design pattern does the WebSocket broadcasting use?

A: Observer/Publish-Subscribe pattern. The server maintains a set of subscribers (`connected_clients`). When new data is available, it publishes (broadcasts) to all subscribers.

Q59: What is MVC architecture?

A: Model-View-Controller. Our project loosely follows this: Model = data structures (SeatZoneState, latest_occupancy), View = frontend (HTML/CSS/JS), Controller = API endpoints and inference loop.

Q60: What is the Single Responsibility Principle in this project?

A: Each component has one job: `_inference_loop` handles AI detection, `_broadcast_loop` handles data distribution, `script.js` handles UI updates, `style.css` handles presentation. They're separate but work together.

Category 10: Data Structures

Q61: What data structures are used?

A: Lists (seat_zones, person_contours), Sets (connected_clients, occupied_seat_ids), Dictionaries (latest_occupancy, seat_last_occupied), NumPy arrays (frames, masks), Union-Find (parent array).

Q62: Why use a Set for connected_clients?

A: Sets provide O(1) add, remove, and membership check. We frequently add/remove WebSocket clients, and the `.discard()` method doesn't raise an error if the element isn't found.

Q63: Why use a Dictionary for seat_last_occupied?

A: Dictionary provides O(1) lookup by seat ID. We need to quickly check when each seat was last seen as occupied — a dictionary maps seat_id → timestamp perfectly.

Category 11: Testing & Debugging

Q64: How would you test this system?

A: Use a pre-recorded video file (set `VIDEO_SOURCE = "329307.mp4"`) for consistent, repeatable testing. Draw seat zones over known positions. Verify occupancy matches what's visible in the video.

Q65: How do you handle camera failure?

A: If `cap.read()` returns `False`, the code either loops the video file (by resetting to frame 0) or waits 0.5 seconds and retries (for a webcam that's temporarily unavailable).

Q66: What happens if the WebSocket disconnects?

A: JavaScript auto-reconnects after 3 seconds. The UI badge shows "Offline". The server removes the stale client from `connected_clients` to avoid sending data to dead connections.

Category 12: General / Counter Questions

Q67: What is a virtual environment and why use it?

A: A virtual environment (`venv`) is an isolated Python installation. Each project can have its own package versions without conflicts. Without it, installing packages for one project could break another.

Q68: What is `requirements.txt` ?

A: A file listing all Python packages the project needs, with version numbers. `pip install -r requirements.txt` installs them all. Ensures anyone can recreate the exact same environment.

Q69: Can this system work at night / in low light?

A: It would need a camera with night vision (infrared). The AI model works on whatever the camera captures — if the image is too dark, detections will fail.

Q70: What is the accuracy of YOLOv8n?

A: YOLOv8n achieves approximately 37.3% mAP (mean Average Precision) on the COCO dataset across all 80 classes. For person detection specifically, it's much higher (~60-70%) since people are a well-represented class.

Q71: How many people can it track simultaneously?

A: There's no hard limit in the software. Practically, YOLOv8n can detect 20-30+ people per frame. The limiting factor is camera resolution and viewing angle — a single camera can realistically monitor 10-20 seats.

Q72: What is Pydantic?

A: A Python library for data validation. When someone sends data to our API, Pydantic automatically checks if it matches the expected format (correct types, required fields). Invalid data is rejected with a clear error message.

Q73: What is the `.pt` file extension?

A: PyTorch model file. It contains the trained neural network weights — millions of numbers that the model learned during training on the COCO dataset.

Q74: What happens if no seats are defined?

A: The system still works — it detects people and shows the video feed. But occupancy counts show "0/0" and the status says "No Seats Defined". The calibrator allows defining seats at any time.

Q75: Could this violate privacy?

A: Yes, video surveillance has privacy implications. In a real deployment, you should: post visible notices about CCTV, not store video recordings, only process frames in memory, and comply with local privacy laws.

> **Tip for Viva:** Don't just memorize answers. Understand the WHY behind each decision. If asked something you don't know, say "That's beyond the current scope, but I would approach it by..." and show thinking ability.

End of Document — Good luck with your presentation! 🎓